

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Военный факультет

Кафедра электронных вычислительных машин

Дисциплина: Жизненный цикл разработки программного обеспечения

ВЕБ-ПРИЛОЖЕНИЕ ДЛЯ СОЗДАНИЯ И КОНТРОЛЯ ЗАДАЧ И
ПРОЕКТОВ.

Студент гр. 334701: А.А. Головатенко

Руководитель: ассистент О.М. Внук

СОДЕРЖАНИЕ

Введение.....	3
1.Проектный раздел	4
1.1. Архитектура приложения.....	4
1.2. Модель данных.....	5
1.3. UML-диаграммы	5
1.3.1. Диаграммы состояний	5
1.3.2. Диаграммы последовательностей	7
1.3.3. Диаграммы активности.....	8
1.4. Макеты интерфейсов	11
2.Реализация проекта	14
2.1. Описание интерфейса и логики работы.....	14
3.Тестирование	14
3.1 План тестирования	14
3.2 Сценарии и акторы.....	15
3.3 Результаты тестирования	15
Заключение	16
Список использованных источников	17

ВВЕДЕНИЕ

В современном мире эффективное управление задачами является одним из ключевых факторов личной и профессиональной продуктивности. Высокий темп жизни, обилие информации и необходимость работать в режиме многозадачности требуют от человека не только дисциплины, но и наличия удобных инструментов для планирования. Стандартные подходы, такие как бумажные ежедневники или простые списки дел, часто не справляются с организацией сложных проектов и не позволяют гибко отслеживать прогресс, что приводит к потере контроля над сроками и снижению мотивации.

В связи с этим возрастает потребность в создании цифровых систем, которые помогают не просто фиксировать задачи, а комплексно управлять ими: группировать по проектам, устанавливать приоритеты, отслеживать статусы выполнения и анализировать общую картину деятельности. Такие инструменты должны быть интуитивно понятными, функциональными и доступными, чтобы стать надёжной опорой в повседневной рутине и достижении долгосрочных целей.

Данный проект посвящён разработке веб-приложения «Todo List», предназначенного для эффективной организации и контроля рабочих и личных дел. В отличие от простых аналогов, приложение позволяет структурировать задачи в рамках проектов, что обеспечивает системный подход к планированию. Пользователи могут легко создавать, редактировать и удалять задачи, отслеживать их выполнение и управлять несколькими проектами одновременно. Цель проекта — предоставить простой, но мощный инструмент для повышения продуктивности и систематизации повседневной деятельности.

1. ПРОЕКТНЫЙ РАЗДЕЛ

1.1. Архитектура приложения

Веб-приложение To-Do List для отслеживания задач построено на монолитной архитектуре, основанной на Spring Boot и паттерне Model-View-Controller (MVC). Такой подход обеспечивает модульность, масштабируемость и эффективное управление компонентами в рамках единого приложения.

Уровень представления, реализованный с использованием шаблонизатора Thymeleaf, предоставляет пользовательский интерфейс. Он включает в себя статические ресурсы (HTML, CSS, JS), которые формируют веб-страницы управления задачами и проектами. Клиент (веб-браузер) взаимодействует с этими страницами, отправляя запросы на сервер для выполнения различных операций.

Бэкенд, разработанный на Spring Boot (Java), обрабатывает запросы, поступающие от уровня представления. Он управляет бизнес-логикой приложения, валидирует данные, а также взаимодействует с базой данных H2 через Spring Data JPA. Контроллеры, такие как TodoController, отвечают за обработку входящих запросов, а репозитории (TodoRepository, ProjectRepository) обеспечивают операции чтения и записи данных.

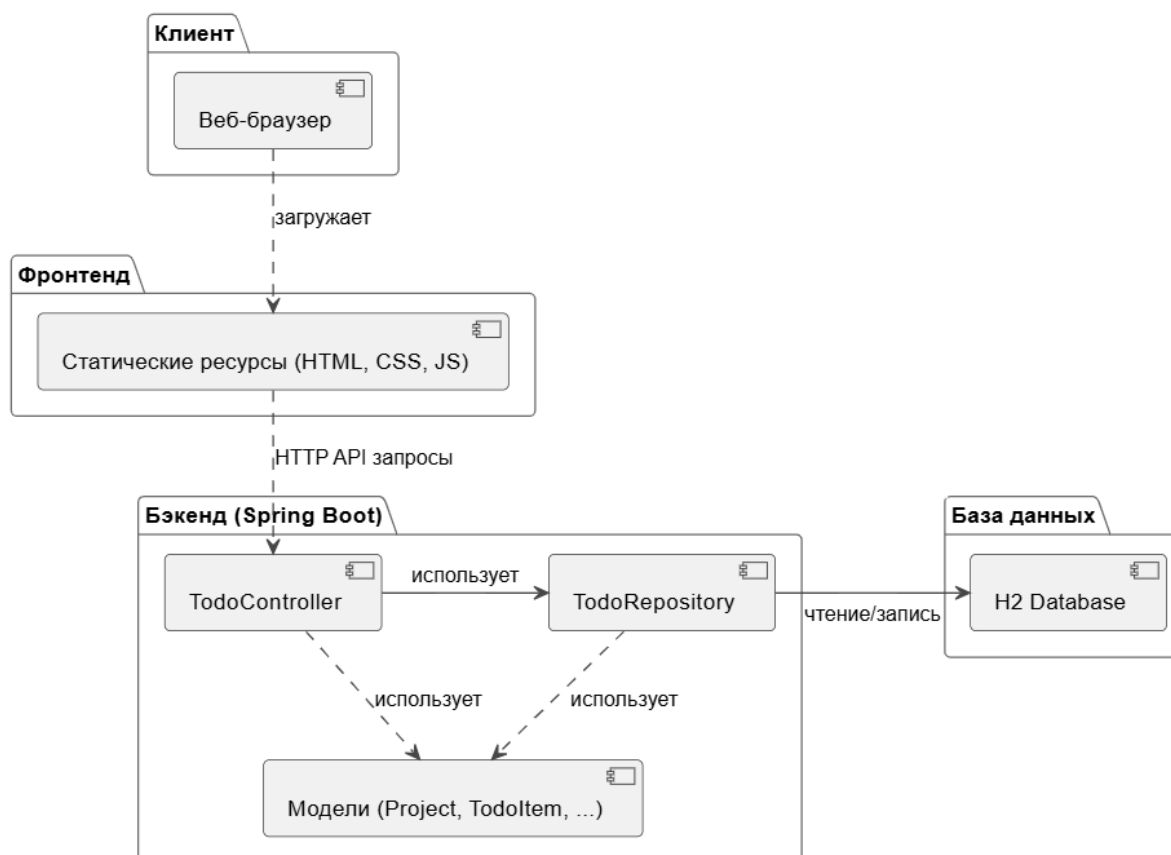


Рисунок 1.1- Диаграмма компонентов

1.2. Модель данных

Основная сущность Project является контейнером верхнего уровня, к которому через связь «один ко многим» привязаны треки задач (TaskTrack). В свою очередь, каждый TaskTrack содержит список конкретных задач (To-Do Item), также связанных отношением «один ко многим». Таким образом, Project представляет собой доску, TaskTrack - её колонки (стадии), а To-Do Item - карточки с задачами.

Таким образом, пользователь управляет проектами, каждый из которых состоит из треков (например, «К выполнению», «В процессе», «Готово»). В эти треки добавляются задачи, по которым отслеживается прогресс выполнения, а система может определять общий статус завершенности как для отдельного трека, так и для всего проекта в целом.

Диаграмма классов представлена на рисунке 1.2.

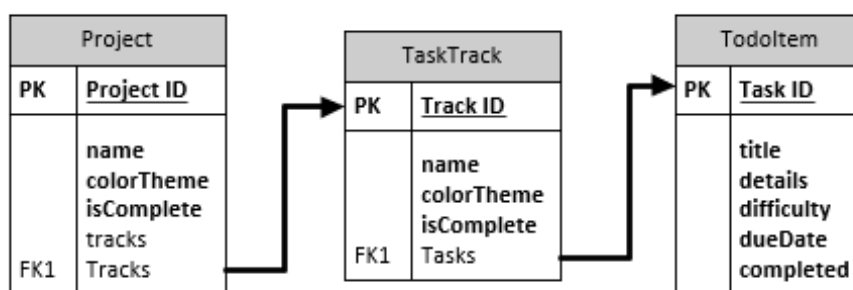


Рисунок 1.2 – Диаграмма классов приложения «To-Do List »

1.3. UML-диаграммы

1.3.1. Диаграммы состояний

На диаграмме состояний проекта представлены все возможные состояния, которые он может принимать в системе: «Выполнен» и «Активен». Переходы между ними отражают логику выполнения проектов, посредством выполнения ветвей. Данная диаграмма помогает описать логику управления проектами.

На диаграмме состояний ветви представлены все возможные состояния, которые она может принимать в системе: «Выполнена» и «Активна». Переходы между ними отражают логику выполнения ветвей, посредством выполнения задач. Данная диаграмма помогает описать логику управления ветвями.

На диаграмме состояний задачи представлены все возможные состояния, которые она может принимать в системе: «Выполнена» и «Активна». Переходы между ними отражают логику выполнения задач, посредством их завершения. Данная диаграмма помогает описать логику управления задачами.

Диаграмма состояний - Project(Проект)



Рисунок 1.3 – Диаграмма состояний проекта

Диаграмма состояний - TaskTrack(Ветвь)

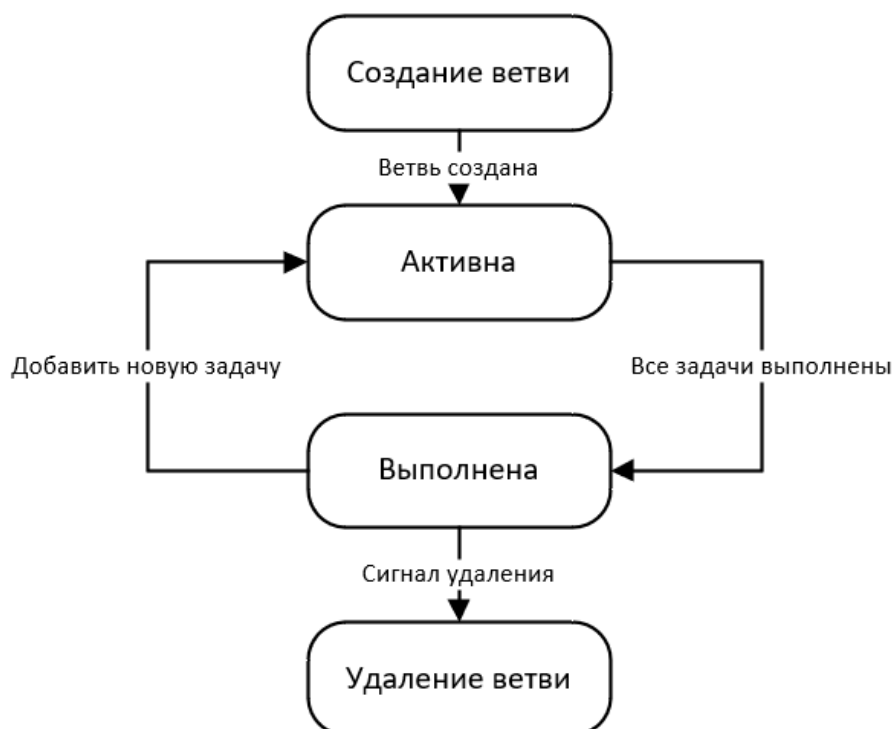


Рисунок 1.4 – Диаграмма состояний ветвей

Диаграмма состояний - Task(Задача)

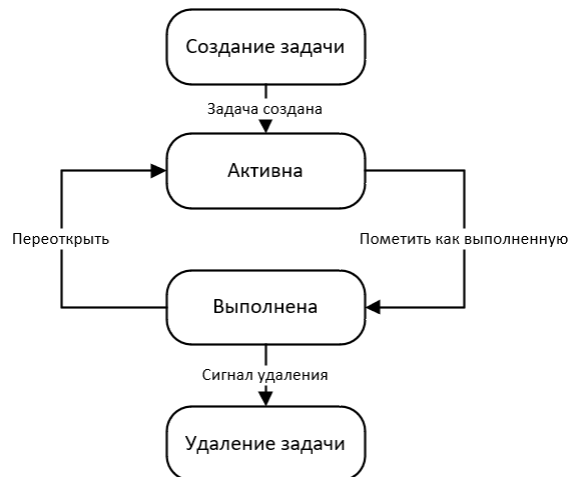


Рисунок 1.5 – Диаграмма состояний задач

1.3.2. Диаграммы последовательностей

Диаграммы последовательностей отражают обмен сообщениями между компонентами To-Do List. Они описывают ключевые сценарии - регистрацию и создание задач.

1. Авторизация пользователя

Пользователь вводит данные, сервер их проверяет, отправляет подтверждение. При ошибке возвращается сообщение о некорректных данных.



Рисунок 1.6 – Диаграмма последовательностей авторизации пользователя

2. Добавление новой задачи

Показан полный цикл добавления задачи: пользователь открывает форму, сервер отображает форму для ввода данных, пользователь вводит данные и отправляет запрос. Сервер проверяет корректность введенных данных, сохраняет запись в базе данных, возвращает подтверждение, после чего обновляется список задач в интерфейсе. Если введенные данные не верны, сервер возвращает ошибку и предлагает ввести данные заново.

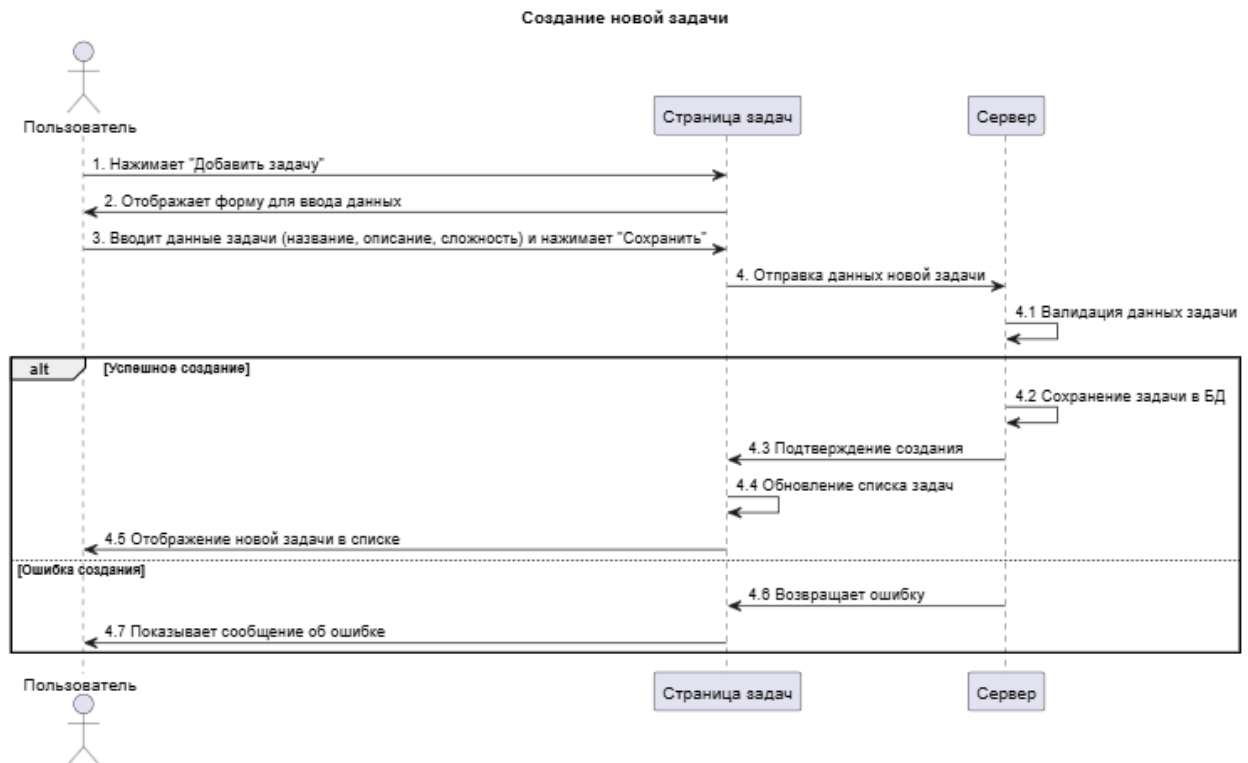


Рисунок 1.7– Диаграмма последовательностей добавления задачи

1.3.3. Диаграммы активности

Диаграммы активности отражают пошаговое выполнение процессов в системе, включая действия пользователя, проверки и результаты. Они важны для проектирования сценариев и логики взаимодействия. В To-Do List такие диаграммы построены для аутентификации, добавления задачи, удаление проекта и фильтрации по сложности. Диаграмма аутентификации показывает: пользователь вводит данные, отправляет их, система их проверяет, при успехе - система переходит на страницу разработчика, при ошибке - предлагает повторить ввод.

Диаграмма добавления задачи показывает: пользователь заполняет форму, отправляет их системе, система отправляет данные на сервер, сервер сохраняет задачу и система обновляет список.



Рисунок 1.8 – Диаграмма процесса авторизации пользователя

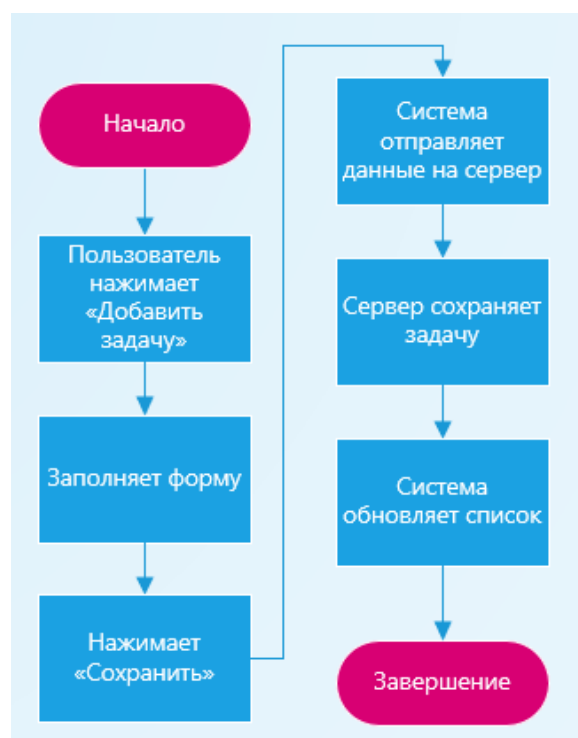


Рисунок 1.9 – Диаграмма добавления задачи

Диаграмма удаления проекта показывает: менеджер выбирает проект, нажимает «Удалить» Система отправляет запрос серверу, сервер удаляет проект, система обновляет список проектов.



Рисунок 1.10 – Диаграмма удаления проекта

Диаграмма фильтрации задач по сложности показывает: пользователь выбирает категорию сложности, сервер возвращает отфильтрованный список задач, и система обновляет список.



Рисунок 1.11 – Диаграмма фильтрации задач по сложности

1.4. Макеты интерфейсов

Для To-Do List были созданы мокапы, чтобы на раннем этапе определить структуру экранов, навигацию и логику взаимодействия без фокуса на дизайн. Цель — обеспечить понятный UX и показать ключевые сценарии: аутентификация, просмотр задач, отслеживание проектов и редактирование проектов.

Главная страница — точка входа для пользователей: просмотр поставленных задач, добавление новых задач, фильтрация задач по сложности, навигация по разделам. Внизу — ссылки на вход и регистрацию.

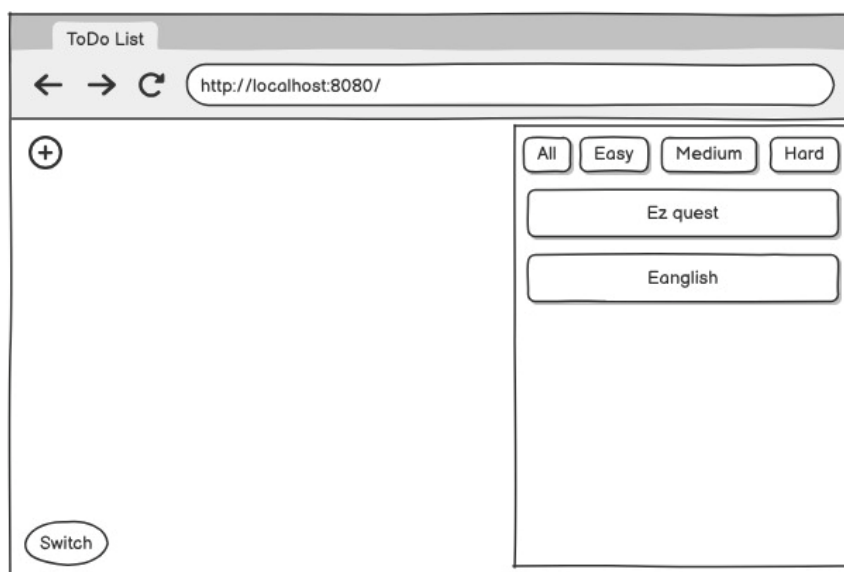


Рисунок 1.12 – Главная страница приложения

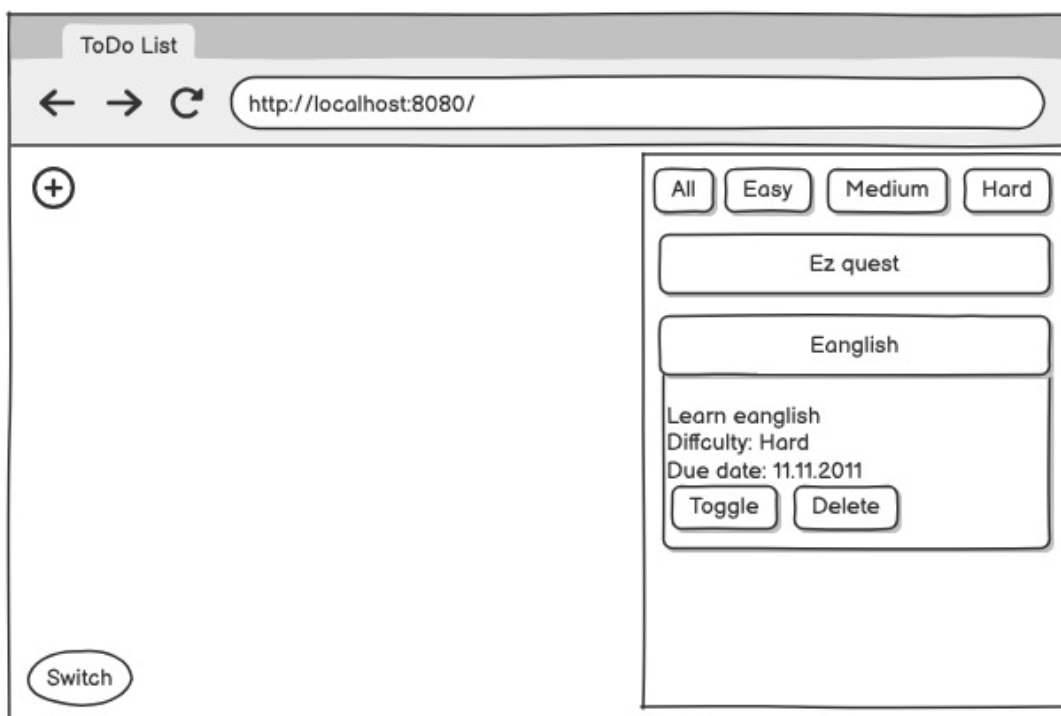


Рисунок 1.13 – Главная страница приложения, просмотр задач

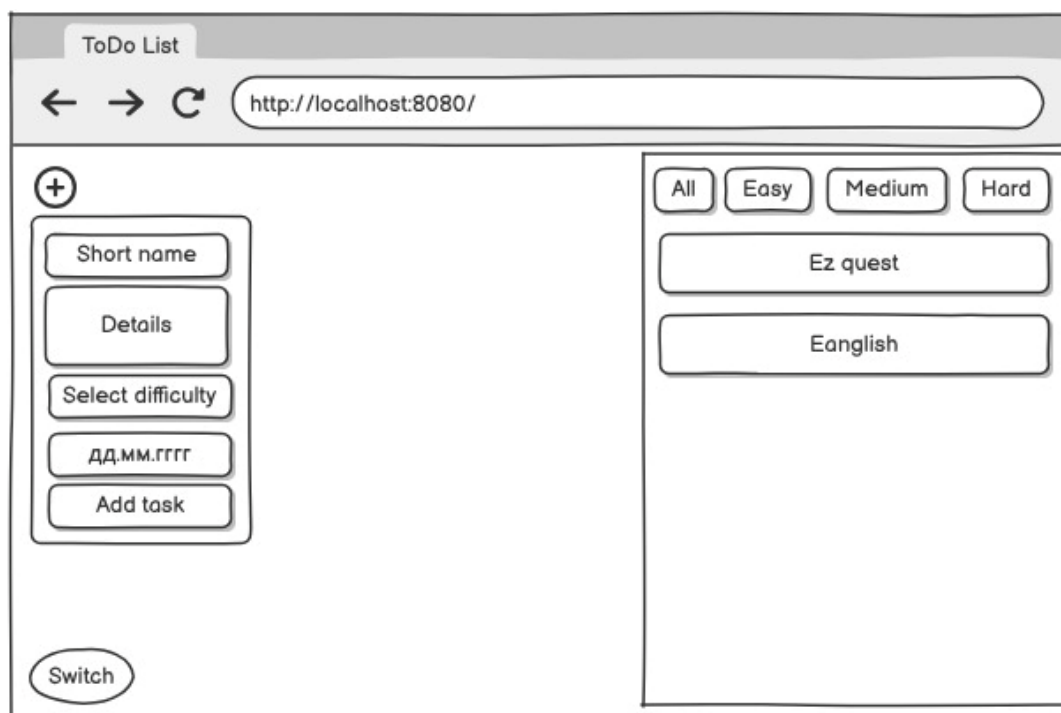


Рисунок 1.14 – Главная страница приложения, добавление задач

На странице проектов пользователь просматривает выполняет проекты, имеет возможность перейти в режим менеджера. На этой же странице менеджер может добавлять, удалять, редактировать проекты. Вверху – ссылка для смены режима, внизу – ссылка для перехода на главную страницу.

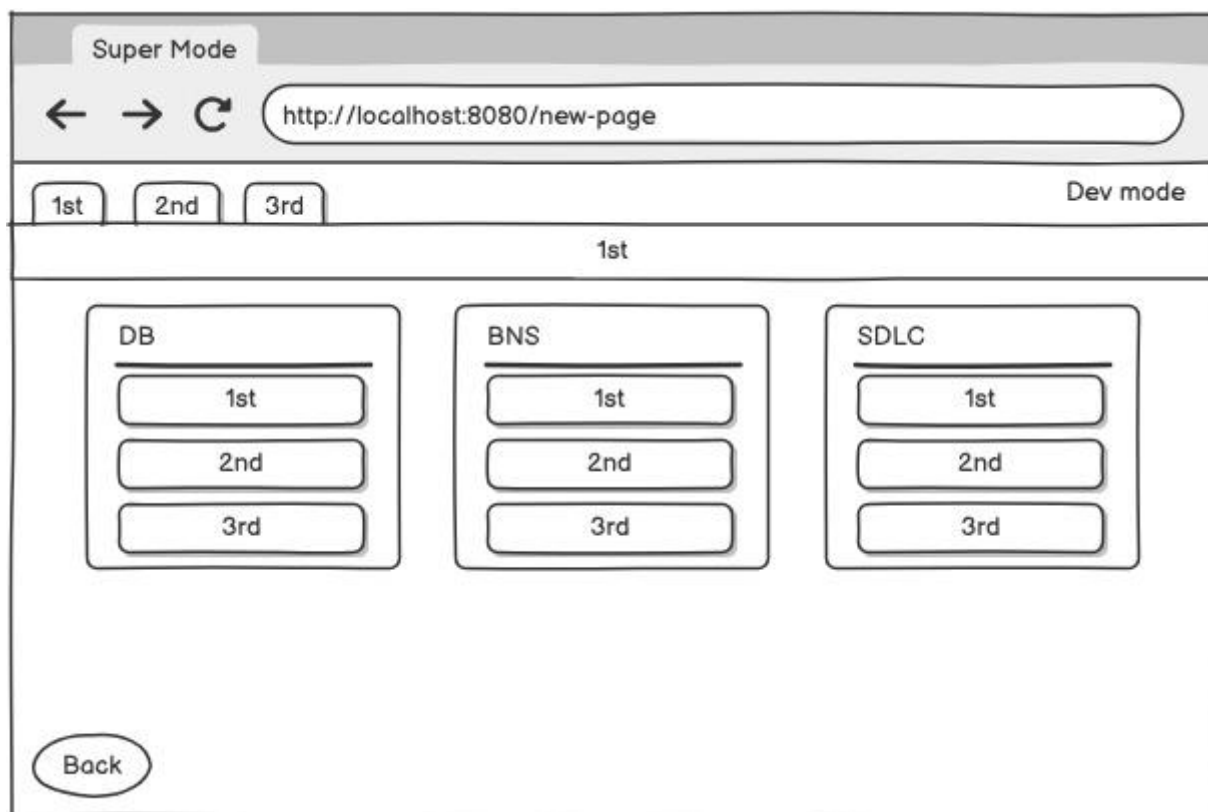


Рисунок 1.15 – Страница проектов (для пользователя)

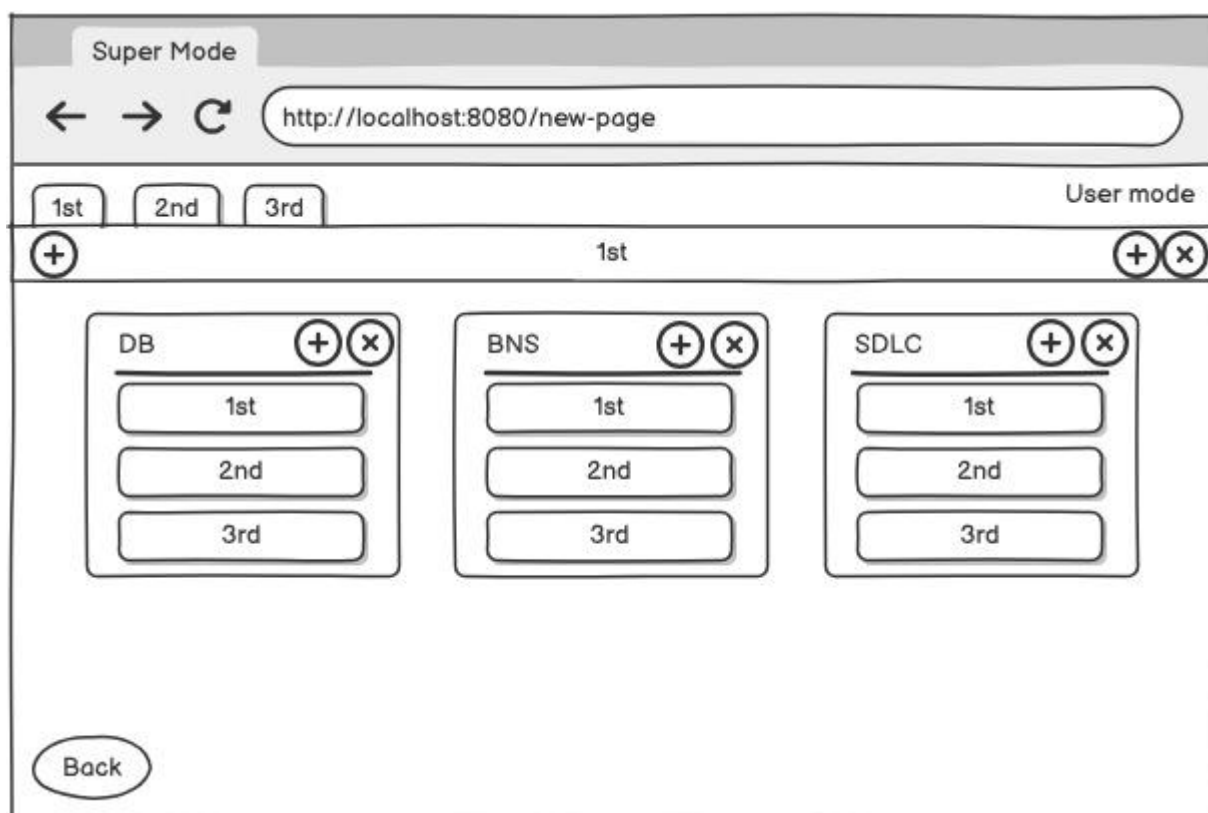


Рисунок 1.16 – Страница проектов (для менеджера)

2. РЕАЛИЗАЦИЯ ПРОЕКТА

2.1. Описание интерфейса и логики работы

Веб-приложение «To-Do List» построено как монолитное приложение, использующее архитектуру Model-View-Controller (MVC). Его бэкенд реализован на Java с применением фреймворка Spring Boot, где контроллеры, такие как TodoController, обрабатывают HTTP-запросы, а репозитории, например ProjectRepository и TodoRepository, обеспечивают взаимодействие с базой данных через Spring Data JPA. Модели Project и TodoItem описывают структуру данных. Фронтенд приложения использует серверный рендеринг: HTML-страницы, такие как todo.html, генерируются на сервере с помощью шаблонизатора Thymeleaf, стилизуются посредством CSS (style.css), а интерактивность обеспечивается клиентским JavaScript (script.js). В качестве базы данных используется встраиваемая H2 Database для хранения информации о проектах и задачах.

Основные пользовательские потоки включают главную страницу, которая служит основным экраном для отображения всех проектов и связанных с ними задач, позволяя пользователю видеть свои дела, их описания и статусы. Пользователь может создавать новые проекты для организации задач, а также добавлять новые задачи внутри каждого проекта, указывая их название и описание, используя для этого отдельную страницу. Приложение также предоставляет функционал для редактирования существующих задач, например, для отметки их выполнения, и их полного удаления, аналогичные операции доступны и для проектов. Пользователь может взаимодействовать с задачами напрямую из списка, например, отмечать их выполнение с помощью чекбоксов, что динамически обновляет их статус. В дальнейшем возможно расширение функционала для фильтрации задач по статусу или проекту, а также добавление функции поиска для быстрого доступа к нужной информации.

3. ТЕСТРОВАНИЕ

3.1 План тестирования

План тестирования приложения «To-Do List» был составлен на основе требований, описанных в SRS, пользовательских сценариев из plan.md и стратегии в TestPlan.md.

Основная цель тестирования заключалась в подтверждении корректной реализации ключевых функций управления задачами и проектами, проверке устойчивости приложения к ошибочным действиям пользователя и гарантии согласованности данных между пользовательским интерфейсом, серверной частью и базой данных.

Объектами тестирования являлись монолитное веб-приложение на Spring Boot, включающее серверную логику на Java, пользовательский

интерфейс, сгенерированный с помощью Thymeleaf, и базу данных H2. В рамках тестирования применялись функциональные, интеграционные и регрессионные тесты, а также проверка валидации данных на уровне бэкенда.

Критериями начала тестирования являлись реализация основных пользовательских сценариев, развёрнутая тестовая среда с базой данных и подготовленные тестовые данные. Критериями завершения - отсутствие критических дефектов, успешное прохождение набора дымовых тестов, а также успешная проверка ключевых сценариев: создание, просмотр, редактирование и удаление проектов и задач, а также изменение статуса задачи.

3.2 Сценарии и акторы

Проверка системы проводилась по сценариям из `plan.md`, охватывающим все ключевые модули и действия пользователей. Особое внимание уделялось функционалу управления проектами и задачами. Если предусмотрена аутентификация, то тестировались регистрация, авторизация и базовая безопасность, включая проверку корректных и ошибочных данных. CRUD-тесты проверяли создание, редактирование, удаление и изменение статуса задач и проектов с корректным обновлением интерфейса и базы данных. Отдельные сценарии касались проверки отображения данных и взаимодействия пользователя с элементами управления.

3.3 Результаты тестирования

Итоговая сводка выполнения тестирования представлена в документе `Test_Results.md`, где зафиксированы статусы прохождения всех сценариев, а также выявленные дефекты с их приоритетами. Всего было выполнено 30 тест-кейсов, из которых 25 успешно прошли, а 5 завершились ошибкой. Среднее время ответа приложения не превышало 250 мс, а операции с базой данных укладывались в 100 мс, что соответствует заявленным нефункциональным требованиям. На основе полученных результатов сделан вывод, что система стабильно функционирует в основных сценариях управления проектами и задачами, корректно обрабатывает пользовательские данные.

ЗАКЛЮЧЕНИЕ

В результате проведённой работы было разработано веб-приложение «Менеджер задач», представляющее собой систему для эффективной организации и контроля выполнения задач. Архитектура приложения построена по классической схеме Model-View-Controller (MVC) как монолитное приложение, что обеспечивает надёжность и простоту поддержки. Бэкенд реализован с использованием Java и Spring Boot, фронтенд - с помощью серверного рендеринга на Thymeleaf, а для хранения данных применяется база данных H2.

Главной особенностью проекта является возможность структурировать задачи в рамках проектов. Такой подход решает основную проблему традиционных списков дел — отсутствие иерархии и невозможность эффективно управлять несколькими целями одновременно. Пользователи могут создавать проекты, наполнять их задачами и отслеживать общий прогресс, что обеспечивает системный подход к планированию.

В процессе реализации были разработаны UML-диаграммы, отражающие внутреннюю логику системы и взаимодействие между её элементами (диаграммы компонентов, состояний, последовательностей и активности). Созданы макеты пользовательского интерфейса, обеспечивающие интуитивное взаимодействие с функционалом приложения. Проведено комплексное тестирование, включающее план, сценарии и анализ результатов, которое подтвердило стабильную работу всех модулей и готовность системы к использованию.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Java - официальная документация.
2. Spring Boot - официальная документация (MVC, Spring Data JPA).
3. Thymeleaf - официальная документация (шаблонизация).
4. HTML, CSS, JavaScript - официальная документация и стандарты W3C.
5. H2 Database - официальная документация.
6. OWASP ASVS / Cheatsheets - рекомендации по безопасности веб-приложений.
7. UML (OMG) - нотации диаграмм компонентов, состояний, последовательностей, активности.